

ASSIGNMENT 20.3

D.CHARMI

2303A51314

BATCH – 05

TASK – 01

Prompt : Generate a Python user registration script that includes password validation. Ensure the password has a minimum length, uses strong rules (uppercase, lowercase, numbers, special characters), rejects weak or common passwords, and uses regular expressions for validation

Code :

```
import re

# Predefined list of common/weak passwords (can be loaded from a file in a real application)
WEAK_PASSWORDS = {"password", "123456", "qwerty", "admin", "user"}

def is_valid_length(password, min_length=8):
    """Checks if the password meets the minimum length requirement."""
    return len(password) >= min_length

def has_required_chars(password):
    """Checks if the password contains uppercase, lowercase, digit, and special characters."""
    # At least one uppercase letter
    if not re.search(r"[A-Z]", password):
        return False
    # At least one lowercase letter
    if not re.search(r"[a-z]", password):
        return False
    # At least one digit
    if not re.search(r"[0-9]", password):
        return False
    # At least one special character (using a common set)
    if not re.search(r"[!@#$%^&*()_~\|{};\:<.>.,?~\-\"]", password):
        return False
    return True

def is_not_common(password, weak_passwords=WEAK_PASSWORDS):
    """Checks if the password is not in the list of common/weak passwords."""
    return password.lower() not in weak_passwords
```

Gemini

D Generate a Python user registration script that includes password validation. Ensure the password has a minimum length, uses strong rules (uppercase, lowercase, numbers, special characters), rejects weak or common passwords, and uses regular expressions for validation

Here's a Python user registration script with strong password validation, including checks for length, character types, and common passwords. I'll provide functions for each validation rule and a main registration function, along with a demonstration.

The user registration script with password

What can I help you build?

+ Gemini 2.5 Flash ▾ ▶

Gemini can make mistakes so double-check it and use code with caution. [Learn more](#)

```
def register_user(username, password):
    """Registers a new user after validating their password."""
    if not is_valid_length(password):
        print("Registration failed: Password must be at least 8 characters long.")
        return False

    if not has_required_chars(password):
        print("Registration failed: Password must contain uppercase, lowercase, a digit, and a special character.")
        return False

    if not is_not_common(password):
        print("Registration failed: Password is too common or weak.")
        return False

    # If all validations pass
    print(f"User '{username}' registered successfully with password: {'*' * len(password)}")
    return True
```

```

# Test cases
print("--- Test Case 1: Valid Password ---")
register_user("john_doe", "SecureP@ss1")

print("\n--- Test Case 2: Too Short Password ---")
register_user("jane_smith", "Short1!")

print("\n--- Test Case 3: Missing Special Character ---")
register_user("alice_w", "NoSpecial1")

print("\n--- Test Case 4: Missing Digit ---")
register_user("bob_j", "NoDigit!")

print("\n--- Test Case 5: Missing Uppercase ---")
register_user("charlie_k", "nouppercase1!")

print("\n--- Test Case 6: Missing Lowercase ---")
register_user("diana_m", "NOLOWERCASE1!")

print("\n--- Test Case 7: Common/Weak Password ---")
register_user("eva_l", "password")

print("\n--- Test Case 8: Another Valid Password ---")
register_user("frank_g", "MyStR0ngP@ssw0rd")

```

Output :

```

--- Test Case 1: Valid Password ---
User 'john_doe' registered successfully with password: *****

--- Test Case 2: Too Short Password ---
Registration failed: Password must be at least 8 characters long.

--- Test Case 3: Missing Special Character ---
Registration failed: Password must contain uppercase, lowercase, a digit, and a special character.

--- Test Case 4: Missing Digit ---
Registration failed: Password must contain uppercase, lowercase, a digit, and a special character.

--- Test Case 5: Missing Uppercase ---
Registration failed: Password must contain uppercase, lowercase, a digit, and a special character.

--- Test Case 6: Missing Lowercase ---
Registration failed: Password must contain uppercase, lowercase, a digit, and a special character.

--- Test Case 7: Common/Weak Password ---
Registration failed: Password must contain uppercase, lowercase, a digit, and a special character.

--- Test Case 8: Another Valid Password ---
User 'frank_g' registered successfully with password: *****
True

```

Explanation : The task checks whether a Python registration script uses strong password validation. It ensures weak passwords are rejected by applying rules like minimum length, character requirements, and security checks.

TASK – 02 :

Prompt : Generate a secure Python (Flask) file upload script. Ensure file type validation, restrict file size, and store files in a safe directory. Prevent malicious uploads by allowing only specific extensions and implementing proper security checks.

Code :

```
#The task evaluates a file upload program for security issues and improves it by adding validations like
allowed file types, size limits, and secure storage to prevent malicious uploads.
from flask import Flask, request, redirect, url_for, send_from_directory
import os
from werkzeug.utils import secure_filename

app = Flask(__name__)

# Configuration
UPLOAD_FOLDER = 'uploads'
ALLOWED_EXTENSIONS = {'txt', 'pdf', 'png', 'jpg', 'jpeg', 'gif'}
MAX_FILE_SIZE = 2 * 1024 * 1024 # 2 MB limit

app.config['UPLOAD_FOLDER'] = UPLOAD_FOLDER
app.config['MAX_CONTENT_LENGTH'] = MAX_FILE_SIZE

# Ensure upload folder exists
os.makedirs(UPLOAD_FOLDER, exist_ok=True)

# Check allowed file types
def allowed_file(filename):
    return '.' in filename and filename.rsplit('.', 1)[1].lower() in ALLOWED_EXTENSIONS

# Upload route
@app.route('/upload', methods=['GET', 'POST'])
def upload_file():
    if request.method == 'POST':
        if 'file' not in request.files:
            return "No file part"

        file = request.files['file']

        if file.filename == "":
```

```

        return "No selected file"

    if file and allowed_file(file.filename):
        filename = secure_filename(file.filename)
        file.save(os.path.join(app.config['UPLOAD_FOLDER'], filename))
        return f"File uploaded successfully! Access it at: /uploads/{filename}"
    else:
        return "Invalid file type"

    return """
    <!doctype html>
    <title>Upload File</title>
    <h1>Upload new File</h1>
    <form method="post" enctype="multipart/form-data">
        <input type="file" name="file">
        <input type="submit" value="Upload">
    </form>
    """

# Route to access uploaded files
@app.route('/uploads/<filename>')
def uploaded_file(filename):
    return send_from_directory(app.config['UPLOAD_FOLDER'], filename)

# Error handling for large files
@app.errorhandler(413)
def file_too_large(e):
    return "File is too large (Max 2MB)", 413

if __name__ == '__main__':
    app.run(debug=True)

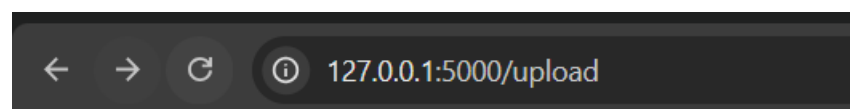
```

Output :

```

JUI Server Instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 542-425-500

```

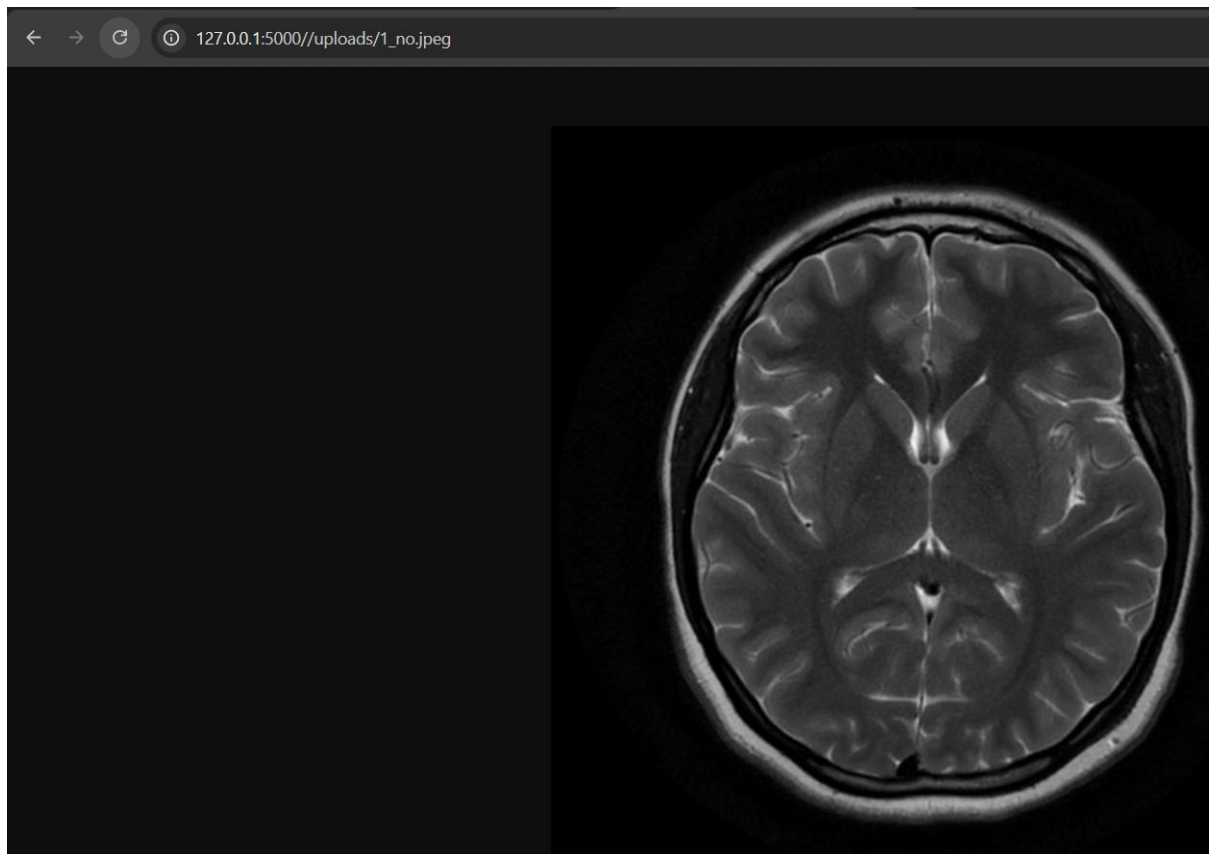


File uploaded successfully! Access it at: /uploads/1_no.jpeg

Upload new File

Choose File 1 no.jpeg

Upload



Explanation : The task evaluates a file upload program for security issues and improves it by adding validations like allowed file types, size limits, and secure storage to prevent malicious uploads.

TASK – 03

Prompt :

#Review the given API endpoint code for security vulnerabilities such as missing authentication, insecure HTTP methods, and lack of rate limiting.

Suggest improvements including token-based authentication, input validation, and rate limiting, and provide a secure version of the API

```
"from flask import Flask, request

app = Flask(__name__)

# Insecure API (No authentication, no validation)

@app.route('/api/data', methods=['GET', 'POST'])
def get_data():

    name = request.args.get('name') # No validation

    return f'Hello {name}'

if __name__ == '__main__':

    app.run(debug=True)"
```

Code :

#Review the given API endpoint code for security vulnerabilities such as missing authentication, insecure HTTP methods, and lack of rate limiting. Suggest improvements including token-based authentication, input validation, and rate limiting, and provide a secure version of the API

```
"from flask import Flask, request

app = Flask(__name__)

# Insecure API (No authentication, no validation)

@app.route('/api/data', methods=['GET', 'POST'])
def get_data():

    name = request.args.get('name') # No validation

    return f'Hello {name}'

if __name__ == '__main__':

    app.run(debug=True)"

# secure_api.py

from flask import Flask, request, jsonify

import time
```

```

app = Flask(__name__)

# ----- CONFIG -----

API_TOKEN = "secure123token"

# simple in-memory storage
users = []

# rate limit storage (IP → request times)
request_log = {}

RATE_LIMIT = 5      # requests
TIME_WINDOW = 60    # seconds

# ----- HOME ROUTE -----

@app.route("/")
def home():
    return """
    <h2>Secure API Running  </h2>
    <p>Available Endpoints:</p>
    <ul>
        <li>POST /add_user</li>
        <li>GET /users</li>
    </ul>
    """

```

```
# ----- AUTHENTICATION -----
```

```
def authenticate(req):
```

```
    token = req.headers.get("Authorization")
```

```
    return token == f"Bearer {API_TOKEN}"
```

```
# ----- RATE LIMIT -----
```

```
def rate_limited(ip):
```

```
    now = time.time()
```

```
    if ip not in request_log:
```

```
        request_log[ip] = []
```

```
    # remove old requests
```

```
    request_log[ip] = [
```

```
        t for t in request_log[ip]
```

```
        if now - t < TIME_WINDOW
```

```
    ]
```

```
    if len(request_log[ip]) >= RATE_LIMIT:
```

```
        return True
```

```
    request_log[ip].append(now)
```

```
    return False
```

```
# ----- INPUT VALIDATION -----
```

```
def valid_user(data):
```

```
    if not data:
```

```

        return False

    if "name" not in data or "email" not in data:
        return False

    if not isinstance(data["name"], str):
        return False

    if "@" not in data["email"]:
        return False

    return True


# ----- ADD USER API -----

@app.route("/add_user", methods=["POST"])
def add_user():

    # authentication check
    if not authenticate(request):
        return jsonify({"error": "Unauthorized"}), 401

    # rate limit check
    if rate_limited(request.remote_addr):
        return jsonify({"error": "Too many requests"}), 429

    data = request.get_json()

    # validation check
    if not valid_user(data):
        return jsonify({"error": "Invalid input"}), 400

```

```

users.append(data)

return jsonify({
    "message": "User added securely",
    "data": data
}), 201

# ----- GET USERS API -----

@app.route("/users", methods=["GET"])
def get_users():

    if not authenticate(request):
        return jsonify({"error": "Unauthorized"}), 401

    if rate_limited(request.remote_addr):
        return jsonify({"error": "Too many requests"}), 429

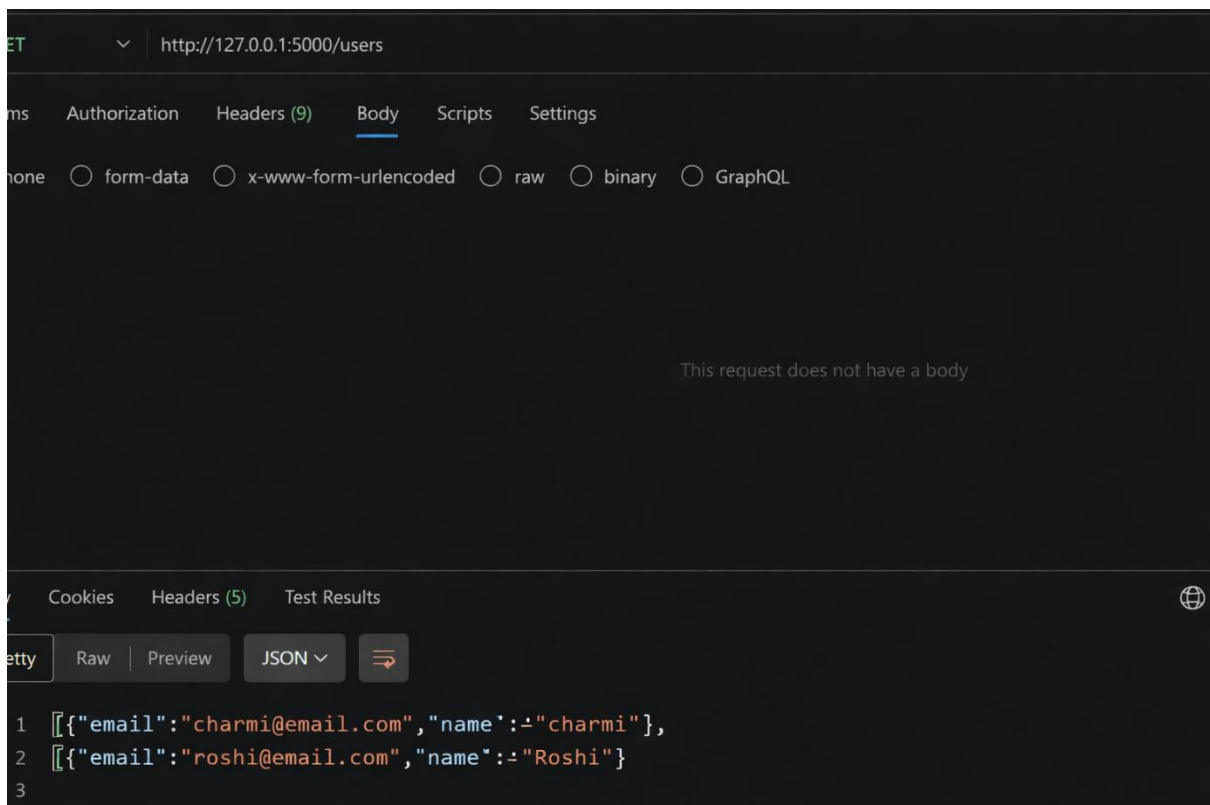
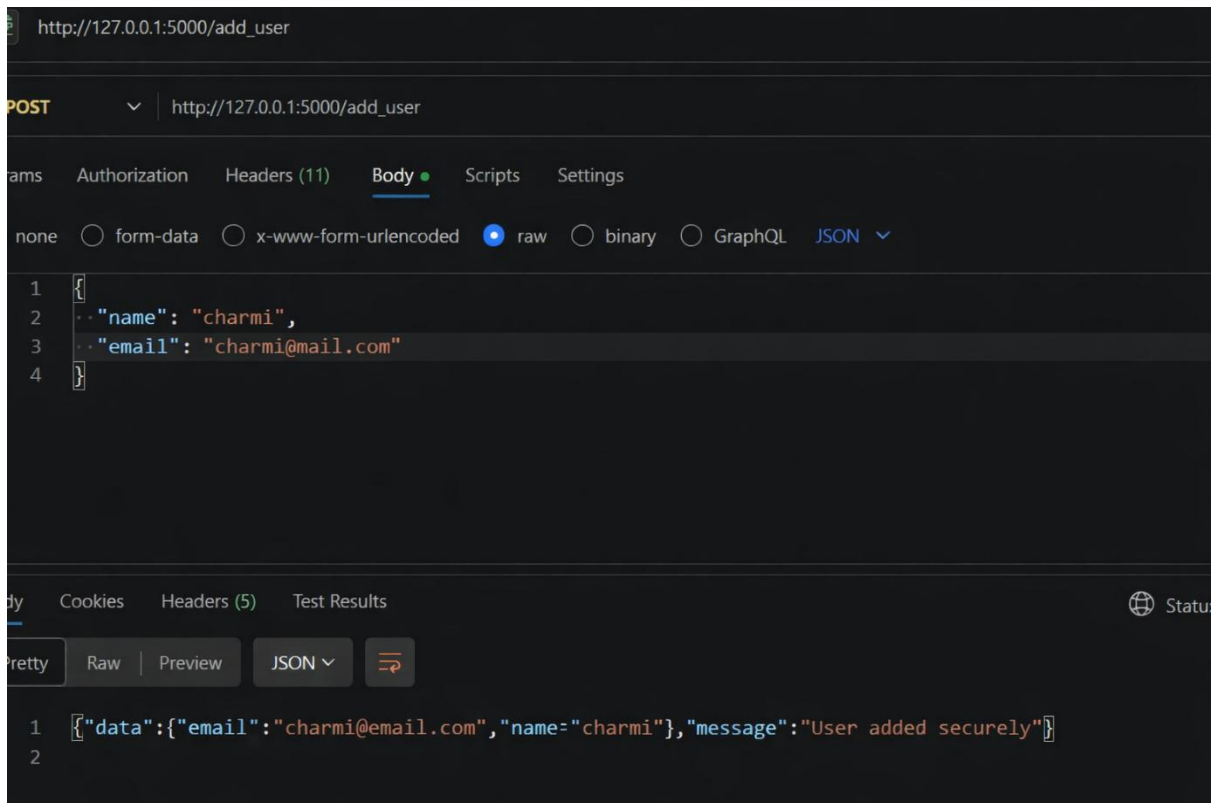
    return jsonify(users)

# ----- RUN SERVER -----

if __name__ == "__main__":
    app.run(debug=False)

```

Output :



Explanation :

The original API had multiple security issues, including the absence of authentication, which allowed anyone to access the endpoint without credentials, the use of insecure HTTP methods where both GET and POST were

permitted without restriction, and a lack of input validation, meaning any type of data could be accepted without checks. In contrast, the secure version improves safety by implementing token-based authentication to ensure only authorized users can access the API, adding rate limiting to control the number of requests from a single user and prevent misuse, and applying input validation to ensure that the data received is in the correct format and contains the required information.

TASK – 04

Prompt : <!--Generate an HTML feedback form with JavaScript that displays user input. Check for XSS vulnerabilities where user input is rendered without escaping. Improve the code by escaping HTML entities and adding security measures like Content Security Policy (CSP) to prevent XSS attacks-->

Code:

```
<!--Generate an HTML feedback form with JavaScript that displays user input. Check for XSS
vulnerabilities where user input is rendered without escaping. Improve the code by escaping HTML
entities and adding security measures like Content Security Policy (CSP) to prevent XSS attacks-->
<!DOCTYPE html>
<html>
<head>
  <title>Secure Feedback Form</title>

  <!-- Content Security Policy -->
  <meta http-equiv="Content-Security-Policy" content="default-src 'self'; script-src 'self'">
</head>
<body>

<h2>Secure Feedback Form</h2>

<form onsubmit="showFeedback(event)">
  <input type="text" id="name" placeholder="Enter your name">
  <button type="submit">Submit</button>
</form>

<h3>Output:</h3>
<div id="output"></div>

<script>
function escapeHTML(str) {
  return str.replace(/&/g, "&amp;")
    .replace(/</g, "&lt;");
}
```

```

        .replace(/>/g, "&gt;")
        .replace(/"/g, "&quot;")
        .replace(/'/g, "&#039;");
    }

function showFeedback(event) {
    event.preventDefault();
    let name = document.getElementById("name").value;

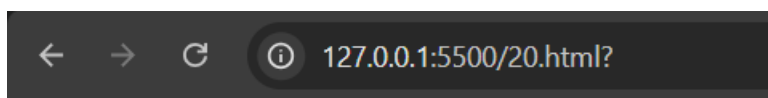
    // ☒ Safe: escaping user input
    let safeName = escapeHTML(name);

    document.getElementById("output").innerHTML = "Hello " + safeName;
}
</script>

</body>
</html>

```

Output:



Secure Feedback Form

Output:

Explanation: This HTML code creates a secure feedback form that prevents Cross-Site Scripting (XSS) attacks. The form allows users to input their name, and when submitted, it displays the input safely by escaping HTML entities. The `escapeHTML` function replaces special characters with their corresponding HTML entities to prevent malicious scripts from being executed. Additionally, a Content Security Policy (CSP) is implemented to restrict the sources of content, further enhancing security against XSS attacks.

TASK – 05

Prompt :

#Generate a Python script using SQLite to fetch user details from a database. Check whether the code is vulnerable to SQL injection (e.g., using string

concatenation). Then refactor the code using parameterized queries (prepared statements) to make it secure."

```
"""import sqlite3

conn = sqlite3.connect("users.db")

cursor = conn.cursor()

username = input("Enter username: ")

# ✗ Vulnerable query (string concatenation)

query = "SELECT * FROM users WHERE username = " + username + ""

cursor.execute(query)

results = cursor.fetchall()

for row in results:

    print(row)

conn.close()"""
```

Code :

```
#Generate a Python script using SQLite to fetch user details from a database. Check whether the code is
vulnerable to SQL injection (e.g., using string concatenation). Then refactor the code using parameterized
queries (prepared statements) to make it secure."
"""import sqlite3

conn = sqlite3.connect("users.db")
cursor = conn.cursor()

username = input("Enter username: ")

# ✗ Vulnerable query (string concatenation)
query = "SELECT * FROM users WHERE username = " + username + ""
cursor.execute(query)

results = cursor.fetchall()

for row in results:
    print(row)

conn.close()"""

import sqlite3
```

```

conn = sqlite3.connect("users.db")
cursor = conn.cursor()

# Create table
cursor.execute("""
CREATE TABLE IF NOT EXISTS users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    username TEXT,
    email TEXT
)
""")

# Insert sample data
cursor.execute("INSERT INTO users (username, email) VALUES (?, ?)", ("charmi",
"charmi@email.com"))
cursor.execute("INSERT INTO users (username, email) VALUES (?, ?)", ("john", "john@email.com"))

conn.commit()
conn.close()

import sqlite3

conn = sqlite3.connect("users.db")
cursor = conn.cursor()

username = input("Enter username: ")

# ☒ Secure query (parameterized)
query = "SELECT * FROM users WHERE username = ?"
cursor.execute(query, (username,))

results = cursor.fetchall()

for row in results:
    print(row)

conn.close()

```

Output :

```

sqlite3.OperationalError: no such table: users
● PS C:\Users\devis\OneDrive\Desktop\AI Assisted Coding> & C:/Users/
OneDrive/Desktop/AI Assisted Coding/20.py"
Enter username: charmi
(1, 'charmi', 'charmi@email.com')
○ PS C:\Users\devis\OneDrive\Desktop\AI Assisted Coding> 

```

Explanation : The original code is vulnerable to SQL injection because it constructs the SQL query using string concatenation, allowing an attacker to manipulate the input and execute arbitrary SQL commands. The refactored code uses parameterized queries (prepared statements) by passing the user input as a parameter to the `execute` method, which prevents SQL injection attacks and ensures that the input is treated as data rather than executable code.